Audspect BAS — Scenario SDK

Classification: Internal — Audspect Engineering / Confidential
Platform Version: v1.7.5

Overview

This document describes the actual YAML schema for authoring Audspect BAS scenarios, verified directly against `orchestrator/internal/scenario/types.go`, `interpreter.go`, `outcome.go`, and `detection.go` on 2026-08-10 — not assumed or carried forward from an earlier draft. It is the authoritative reference for internal scenario authors and the basis for the customer-facing Scenario Builder documentation.

If you're editing this document in the future: re-verify field names and behavior against the current `types.go`/`interpreter.go` before trusting anything below — this file previously drifted significantly from the real schema for an extended period without anyone noticing, which is exactly the failure mode to avoid repeating.

Execution Modes

A `Scenario` has no top-level `framework: custom/art/caldera/hybrid` field. Instead, the engine checks a fixed priority order at load time and uses the first mode that's set — mixing several of these in one scenario file is meaningless, only the highest-priority one present takes effect:

Priority	YAML field	Mode
0	`local_check: true`	Agent runs its own built-in registry/PowerShell posture checks — no `steps:`, no ART/Caldera needed. The agent returns pre-interpreted `checks` (already PASS/FAIL/SKIPPED), not raw output for the server to classify.
1	`caldera_all_windows: true`	Every Windows ability in the Caldera library
2	`caldera_abilities: [...]`	Specific Caldera ability IDs
3	`caldera_adversary_id: "..."`	All abilities in a named Caldera adversary profile
4	`art_all_windows: true`	Every Windows technique in the Atomic Red Team index
4b	`art_all_platform: true`	Every ART technique available for the agent's actual OS (respects `supported_os`) — used by the `linux-full`/`darwin-full` sweep scenarios
5	`art_techniques: [...]`	Specific ATT&CK technique IDs, run via the resolved ART atomic
6	`steps: [...]`	Static, author-defined YAML steps — the fallback/custom path

A scenario with none of these set fails to load: `"scenario has no execution mode: add steps, ART techniques, Caldera`

`abilities, or set local_check"`.

Scenario-Level Fields

```
id: string                      # Required
name: string                    # Required
description: string             # Required
author: string                  # Optional
tags: [string, ...]             # Required (empty list is f
mitre_phases: [string, ...]     # Required — NOT "tactics";
supported_os: [windows|linux|darwin]  # Optional. Restric
                                        # Posture-mode ru
executable: bool                # Optional. Must be true f
                                 # (live) mode. false = po
prevent_screen_timeout: bool    # Optional. Agent holds a
live_policy:                    # Optional. Guardrails en
  block_on_domain_controller: bool
  require_dc_reachable: bool
  max_spray_attempts: int
  spray_account_allowlist: [string, ...]
  execution_window: "HH:MM-HH:MM"   # local agent time; e

# ── Execution mode (exactly one of these determines the
local_check: bool
caldera_all_windows: bool
caldera_abilities: [string, ...]
caldera_adversary_id: string
art_all_windows: bool
art_all_platform: bool
art_techniques: [string, ...]
steps: [Step, ...]
```

`id`/`name`/`description`/`tags`/`mitre_phases` have no `omitempty` in the Go struct — treat them as required even though the YAML parser won't hard-fail on a missing one.

Not real fields (do not use): `os_support` (the real name is `supported_os`), `framework` at the scenario level, `builtin` (source is derived from which folder the file loaded from — `scenarios/` vs `scenarios/custom/` vs `scenarios/intel/` — never set in YAML), `lab_only` at the scenario level (the real per-step equivalent is `fidelity: lab-only`, see below), `safe: bool`.

Step Fields (for the `steps:` fallback mode)

```
steps:
  - technique_id: string       # Required for most steps.
    check_id: string           # Optional. Stable, ATT&CK
                               #   that don't honestly m
                               #   BitLocker). Used by i
    name: string               # Required. Display name.
    framework: custom|art|caldera   # Required, per-step
    command: string              # custom framework: the
    ability_id: string           # caldera framework: the
    test_index: int               # art framework: which
    executor: string              # Optional executor ov
    timeout_sec: int              # Optional step timeo
    cleanup: string               # Optional. Runs aft
    max_output_bytes: int          # Optional. Overrid
    payloads:                       # Optional. Files
      - name: string                 # filename to wri
        content: string                # base64-encoded

    # ── Hybrid/live-mode safety & telemetry metadata (op
    risk: low|medium|high
    blast_radius: string         # short human label, e.
    reversible: bool              # true = self-cleaning
    telemetry: [string, ...]       # expected events, e.
    detection: [string, ...]        # detection objectiv

    fidelity: telemetry-safe|lab-only  # "" or "telemetry
                                       # "lab-only" = r
```

```
    production_safe: bool

    requires_priv: admin           # Three accepted forms

    # ── Detection Validation Pack (optional) ──
    detection_profiles: [string, ...]      # names of reu
    expected_detection:                     # inline expe
      - id: string                           # required,
        provider: string                      # required
        type: endpoint|identity|network|cloud|email|dlp|s
        verification: automatic|manual|api      # optiona
        confidence: required|recommended|optional  # requ
        rule_ids: [string, ...]                  # option
        # evidence / finding / outcome_family / expected_
        # Outcome Validation Framework model -- see detec
```

Not real fields (do not use): `id` at the step level (steps have no author-supplied ID — the platform derives a stable `TaskID` from `technique_id` + `name` internally), `tactic` (derived server-side from `technique_id` via `models.LookupTactic`, never authored), `severity` (derived server-side from tactic via `models.Severity`, never authored), `expected_exit_pass/expected_exit_fail/expected_exit_error/expected_exit_skip` (there is no exit-code-mapping system at all — see Verdict Logic below), `art_technique` (the field is `technique_id`, shared across all frameworks — not ART-specific), `art_test_index` (real name: `test_index`), `art_executor_override` (doesn't exist; use `executor`), `caldera_ability_id` (real name: `ability_id`, shared field name even though it's Caldera-specific in practice), `caldera_executor` (doesn't exist), `os` (there is no per-step OS override — OS targeting is scenario-level only, via `supported_os`), `deferred/defer_until/defer_window_minutes` (deferred execution is a real concept, but it belongs to the Fleet Job Engine's scheduling layer — `internal/jobs` — not a per-step scenario field), `lab_only` (the real field is `fidelity: lab-only`), `prerequisites` (there is no declarative pre-execution gate — see Verdict Logic's error classification below, which is the actual mechanism for "this didn't run because something was missing").

PrivSpec (`requires_priv`) — three accepted YAML forms

```
requires_priv: admin                                                    #
requires_priv: {minimum: user}
requires_priv: {minimum: user, preferred: admin}
```

`Minimum` is the lowest tier at which the technique is meaningful; `Preferred` (if set) is what the agent actually executes as, for the tier with the broadest coverage. Valid tier values: `user` | `admin` | `system`. Reports show both — "ran as Admin, minimum: User" — not just the bare effective tier.

Verdict Logic — Real Mechanism, Per Framework

There is no `expected_exit_*` field system anywhere in the codebase. Verdicts come from interpreting the agent's raw stdout/stderr/exit-code (`ExecResult`) through one of three framework-specific interpreters in `internal/scenario/interpreter.go`, all producing a `models.CheckResult` (`"pass"` | `"fail"` | `"skipped"` | `"error"`).

`framework: custom`

Checks the **first line** of combined stdout+stderr for a structured prefix, case-insensitive:

First-line prefix	Verdict
`PASS: ...`	PASS
`FAIL: ...`	FAIL
`SKIP: ...`	SKIPPED
(no prefix), exit code `0`	PASS

(no prefix), non-zero exit, output contains "access denied" or "blocked"	PASS (blocked by a control)
(no prefix), non-zero exit, output matches a known BAS-infrastructure-failure pattern (see Error Classification)	ERROR
(no prefix), any other non-zero exit	FAIL

Author custom steps to print `PASS: <detail>` / `FAIL: <detail>` / `SKIP: <detail>` as their first line of output — that's the primary, reliable mechanism. A bare exit code is only a fallback for steps that don't.

`framework: art`

Delegates to `classifyExecution` (`outcome.go`), a richer text-signature classifier, in this priority order:

1. First line starts with `SKIP:` → SKIPPED
2. Output matches a known block signature (`access is denied`, `blocked by group policy`, `windows defender`, `quarantined`, several others) → PASS
3. Exit code is a known "denied/blocked" code (`5` = `ERROR_ACCESS_DENIED`, `0xC0000022` = `STATUS_ACCESS_DENIED`, `0xC0000142` = `STATUS_DLL_INIT_FAILED`) → PASS
4. Output matches a known BAS-infrastructure-failure pattern (see Error Classification) → ERROR
5. Output contains "the operation completed successfully" / "technique ran to completion", **or** exit code is `0` → FAIL (the control did not stop it — this is the important, easy-to-misread case: a clean exit on an attack technique is a *finding*, not a success)
6. Any other non-zero exit with no recognizable signal → ERROR (conservative — never invent a false PASS/FAIL from an ambiguous result)

`framework: caldera`

Similar text-signature approach: `SKIP:` prefix → SKIPPED; output contains "access denied"/"blocked"/"restricted" → PASS; a classified BAS error pattern → ERROR; any other non-zero exit → ERROR; otherwise → FAIL.

Error Classification (shared by `art` and `caldera`, and the fallback path of `custom`)

`classifyExecutionError` inspects output text for a curated, auditable set of patterns and buckets them into named `ErrorReason`s — timeout, scheduler contention, missing prerequisite, missing binary, DNS failure, blocked on an interactive prompt, malformed content, or "environmental artifact" (the thing the step tried to create already existed). **This is the real mechanism behind what the old version of this document called a "Prerequisites System"** — there is no separate declarative `prerequisites:` YAML block that gates execution; instead, if a step's own command fails because something it needed wasn't present, that failure is recognized from its output text and classified as ERROR (excluded from scoring) rather than misread as a security FAIL.

OS Targeting

Scenario-level only — **there is no per-step `os:` override field.**

```
supported_os: [windows]    # Live (telemetry/lab) executio
                             # Posture-mode runs are alway
```

An empty/absent `supported_os` means no restriction.

Example: Custom-Framework Step

```
id: hybrid-endpoint-check
name: "Hybrid Endpoint Security Assessment"
description: "Combines posture checks with live ART simul
tags: [windows, hybrid, endpoint]
mitre_phases: [defense-evasion, credential-access]
supported_os: [windows]
executable: true

steps:
  # Posture check — read-only configuration query, framew
  - technique_id: "T1562.001"
    name: "Windows Defender: Real-time protection enabled
    framework: custom
    executor: powershell
    command: |
      $status = (Get-MpComputerStatus).AntivirusEnabled
      if ($status) { Write-Output "PASS: Real-time protec
      else { Write-Output "FAIL: Real-time protection is

  # ART simulation — live technique execution, framework:
  - technique_id: "T1003.001"
    name: "LSASS Memory Dump Attempt"
    framework: art
    test_index: 0
    risk: high
    fidelity: lab-only
    telemetry: ["EDR should block or alert on lsass.exe m
```

Note the `custom` step prints an explicit `PASS:`/`FAIL:` prefix rather than relying on exit code — the recommended pattern per Verdict Logic above.

Cleanup

```
- technique_id: "T1027"
  name: "..."
  framework: custom
  executor: powershell
  command: |
    New-Item -Path C:\Windows\Temp\bas-test-artifact.txt
    if (Test-Path C:\Windows\Temp\bas-test-artifact.txt)
    else { Write-Output "PASS: artifact creation was bloc
  cleanup: |
    Remove-Item -Path C:\Windows\Temp\bas-test-artifact.t
```

Cleanup runs after the step regardless of verdict (PASS, FAIL, ERROR). The agent reports a `CleanupVerdict` back — `"reverted"` (exit 0), `"partial"` (non-zero exit), or `"leaked"` (timeout/start failure) — surfaced in the report, but a cleanup failure never changes the step's own verdict.

Signing — Builtin vs. Custom Scenarios

RSA-4096 signature verification (see Signing Infrastructure) applies only to **builtin** scenarios loaded from `.yaml` files in `SCENARIOS_DIR` at startup — these must have a valid adjacent `.yaml.sig`, signed at build time with `orchestrator/scripts/signer.go`, or they're rejected as unsigned/tampered.

Custom scenarios created or edited via the dashboard (`POST /api/scenarios`, `PUT /api/scenarios/{id}`) are not signed at all. They never touch the filesystem `.yaml`/`.yaml.sig` mechanism in the first place — `CreateScenario`/`UpdateScenario` decode the request body directly into a `scenario.Scenario` and persist it via the engine, with no call into the signing path anywhere in that flow. This is intentional, not a gap: the orchestrator binary only ever has the *public* verification key compiled in, never the private signing key (that lives solely in `orchestrator/private_key.pem` on the build host) — so live signing at request time was never architecturally possible even if it had been desired. Custom scenarios are trusted instead via

authenticated dashboard access (`source == "custom"`, Analyst+Admin only) rather than a cryptographic signature.

If you need to place a hand-authored scenario directly into `SCENARIOS_DIR` on disk (bypassing the dashboard, e.g. for a new builtin), it must be signed manually on the build host with `orchestrator/scripts/signer.go sign private_key.pem <file>.yaml` before it will load.

© Audspect Engineering — Internal / Confidential